
title: "Agent Provenance, Intent Binding, and Capability Attenuation for Autonomous Multi-Agent Systems (CodeTether Profile)" abbrev: "Agent Provenance" docname: draft-seaburg-codetether-agent-provenance-00 category: std ipr: trust200902 area: Security workgroup: WIMSE keyword:

- agent
- identity
- oauth
- provenance
- attestation
- a2a
- mcp

author:

- name: Riley Seaburg organization: (Independent) email: riley@seaburg.dev

normative: RFC2119: RFC6749: RFC7519: RFC8174: RFC8693: RFC8705: RFC9449: RFC9700:

informative: I-D.ietf-wimse-arch: I-D.ietf-wimse-workload-creds: I-D.goswami-agentic-jwt: I-D.klrc-aiagent-auth: I-D.oauth-ai-agents-on-behalf-of-user: I-D.ni-wimse-ai-agent-identity: I-D.messous-eat-ai: I-D.rosenberg-cheq: SPIFFE: title: "SPIFFE: Secure Production Identity Framework for Everyone" target: <https://spiffe.io> A2A: title: "Agent2Agent (A2A) Protocol Specifications" target: <https://a2a-protocol.org> MCP: title: "Model Context Protocol Specification" target: <https://modelcontextprotocol.io>

— abstract

This document defines an OAuth 2.0 and WIMSE-compatible profile for autonomous multi-agent systems that addresses four gaps left open by existing specifications: (1) cryptographic binding of user intent to non-deterministic agent execution across multi-turn sessions; (2) information-provenance taint propagation as a first-class, monotonic token claim; (3) capability-attenuated sub-agent spawning with enforceable depth and fan-out limits; and (4) cross-trust-domain agent-to-agent invocation with verifiable attestation handoff. The profile is derived from operational experience with CodeTether,

an autonomous multi-agent development system, and is designed to compose with RFC 8693 token exchange, the WIMSE architecture, and the Agentic JWT draft rather than replace them.

— middle

Introduction

The rapid deployment of autonomous AI agents in production environments has outpaced the identity and authorization protocols available to secure them. {{RFC8693}} provides token exchange semantics sufficient for static service-to-service delegation, but its threat model does not contemplate actors that select their own downstream invocations based on model inference, ingest attacker-controllable content that may influence those invocations, or dynamically spawn sub-actors whose capabilities must be bounded.

Multiple in-flight drafts address subsets of these problems. {{I-D.ietf-wimse-arch}} provides workload identity primitives applicable to AI intermediaries. {{I-D.goswami-agentic-jwt}} introduces agent checksums and workflow binding to solve what it terms the “intent-execution separation problem.” {{I-D.klrc-aiagent-auth}} composes WIMSE, SPIFFE, and OAuth for the agent case. None of these, individually or in combination, provide a complete solution to the four gaps enumerated in the abstract.

This document defines the CodeTether Profile, which specifies:

- A claim schema (Section 4) that extends the Agentic JWT with provenance-tracking fields suitable for multi-turn agent sessions.
- A monotonic taint propagation mechanism (Section 5) enforceable at every resource server and policy decision point.
- A sub-agent spawn protocol (Section 6) that cryptographically attenuates capabilities at each delegation hop and enforces bounded recursion.
- A cross-trust-domain invocation profile (Section 7) for agent-to-agent calls that span organizational identity boundaries.
- A tool-call attestation mechanism (Section 8) that binds individual tool invocations to a specific model output and system configuration.

The profile is informed by operational experience with CodeTether, a production autonomous multi-agent system whose characteristics are described in Appendix A. The profile is not specific to CodeTether and is intended for any multi-agent system with comparable properties.

Requirements Language

{{::boilerplate bcp14-tagged}}

Terminology

This document uses terminology from {{RFC8693}}, {{I-D.ietf-wimse-arch}}, and {{I-D.goswami-agentic-jwt}}, and adds the following terms.

Human Principal: : The natural person on whose ultimate authority an agent acts. Identified by a persistent subject identifier in a human-identity trust domain.

Agent Archetype: : A class of agent defined by its model identifier, system prompt hash, tool manifest hash, and configuration hash. Multiple agent instances MAY share an archetype.

Agent Avatar: : A persistent, named agent instance bound to a specific Human Principal or organizational principal. An avatar has a stable identifier across sessions and MAY be instantiated as any archetype for which it is authorized.

Agent Session: : A bounded interval during which an avatar executes one or more turns of reasoning in pursuit of a task. A session has a single originating intent, a monotonic taint state, and a consumed-capability budget.

Agent Turn: : A single model inference within a session that MAY emit zero or more tool-call requests or sub-agent spawn requests.

Taint: : A monotonic set of provenance markers attached to a session, recording every untrusted information source that has entered the reasoning context. Taint is never reduced within a session; once set, it constrains all subsequent authorization decisions in that session.

Capability Envelope: : The maximum set of tool invocations, resource accesses, and sub-agent spawn rights that a token authorizes. Capability envelopes are specified as a Rego-compatible attribute set and attenuate monotonically down a delegation chain.

Spawn Depth: : The number of sub-agent delegation hops between a session and a root human principal. The root human session has spawn depth 0.

Fan-out: : The number of concurrent sub-agents a given agent session has spawned at a given moment.

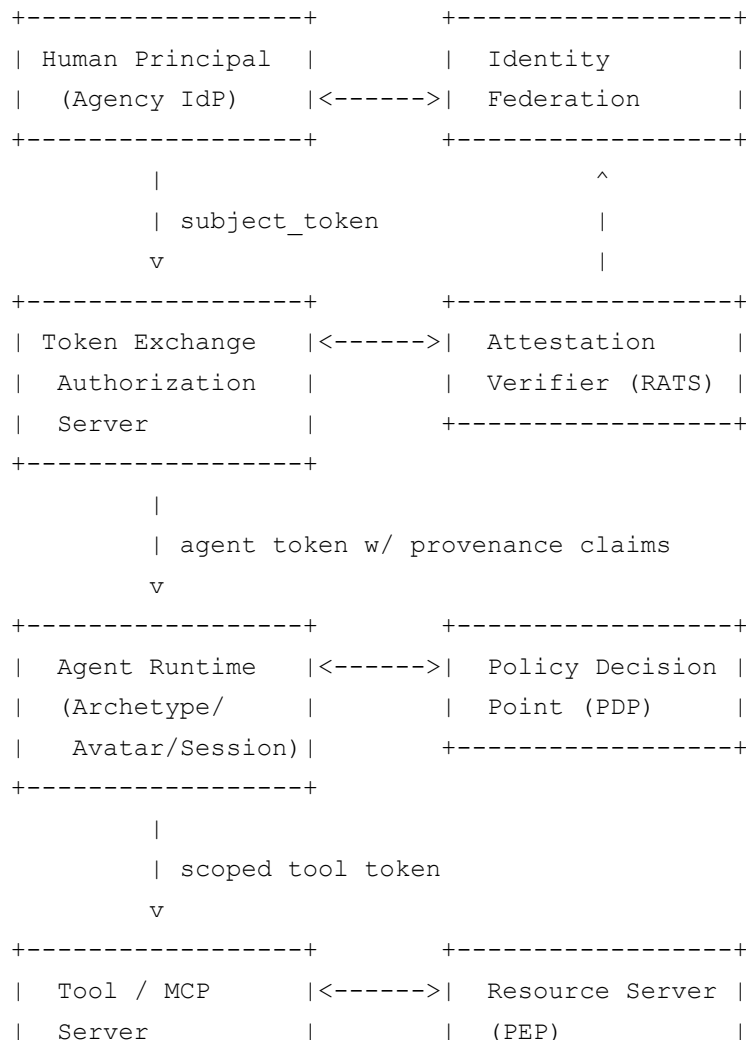
Attestation Quote: : A signed assertion by a trusted runtime (e.g., a confidential-computing enclave, a hosted inference provider) binding a model output to a specific model identifier, system prompt hash, and execution environment measurement.

Architecture Overview

The CodeTether Profile composes with, rather than replaces, the following layers:

- Transport authentication via mTLS with WIMSE Workload Identity Certificates ({{I-D.ietf-wimse-workload-creds}}) or DPoP {{RFC9449}}.
- Token issuance via extended {{RFC8693}} token exchange.
- Token format extending {{I-D.goswami-agentic-jwt}} with the claims defined in Section 4 of this document.
- Policy decision via an external Policy Decision Point (PDP), typically implementing an attribute-based access control model such as Rego.

The four-layer identity hierarchy (Human -> Archetype -> Avatar -> Session) is represented in tokens via a deterministic claim structure (Section 4.1) that allows every PDP in the chain to reconstruct the full delegation context from a single token without out-of-band lookups.



+-----+

+-----+

{: title="CodeTether Profile Component Model"}

Token Claim Schema

Core Claims

A CodeTether Profile agent token is a JWT ({{RFC7519}}) that **MUST** contain the following claims in addition to those required by {{RFC7519}}:

`iss`, `aud`, `exp`, `iat`, `jti`, `sub` :: Standard {{RFC7519}} claims. `sub` **MUST** be the avatar identifier.

`act` :: Actor claim as defined in {{RFC8693}} Section 4.1. The `act` chain **SHALL** represent the full delegation path from the immediate caller back to the root human principal. Implementations **MUST** reject tokens whose `act` chain cannot be fully verified.

`azp` :: Authorized party, set to the archetype identifier.

`ctp_version` :: String. The CodeTether Profile version this token conforms to. This document defines version "1" .

`ctp_avatar` :: Object. Avatar metadata:

- `id` : Stable avatar identifier (URI, typically a SPIFFE ID).
- `owner` : The human principal subject identifier.
- `class` : Avatar classification tier (implementation-defined; e.g., "research" , "coder" , "procurement").
- `created_at` : ISO 8601 timestamp of avatar creation.

`ctp_archetype` :: Object. Archetype binding:

- `id` : Archetype identifier (URI).
- `model` : Model identifier (e.g., "claude-opus-4-7").
- `model_provider` : Provider identifier.
- `system_prompt_hash` : SHA-384 hex of the canonicalized system prompt.
- `tool_manifest_hash` : SHA-384 hex of the canonicalized tool manifest presented to

the model at session start.

- `config_hash` : SHA-384 hex of additional configuration (temperature, sampling parameters, etc.).

`ctp_session` : : Object. Session binding:

- `id` : Unique session identifier (UUIDv7 RECOMMENDED).
- `origin_intent_hash` : SHA-384 hex of the canonicalized initial user request that originated the session. Does not change across turns.
- `turn` : Integer. Monotonically increasing turn counter within the session. Starts at 1.
- `parent_jti` : JTI of the prior turn's token in the same session. Absent for turn 1.
- `started_at` : ISO 8601 timestamp of session start.

`ctp_capability` : : Object. Capability envelope (see Section 6):

- `tools` : Array of tool identifiers authorized for invocation.
- `tool_constraints` : Object mapping tool identifier to argument constraints expressed as a JSON Schema or Rego-compatible predicate.
- `budget` : Object with `max_tool_calls` , `max_input_tokens` , `max_output_tokens` , `max_wall_clock_seconds` .
- `spawn` : Object with `max_depth` , `max_fanout` , `allowed_archetypes` .

`ctp_taint` : : Array of taint markers (see Section 5). MUST only grow across turns within a session.

`ctp_attestation` : : Object. Optional attestation quote (see Section 8). RECOMMENDED for any token whose capability envelope includes tools classified as sensitive by local policy.

`cnf` : : Confirmation claim per {{RFC7800}}. REQUIRED. Either a `jkt` for DPOP binding or `x5t#S256` for mTLS binding.

Claim Canonicalization

All hash claims (`system_prompt_hash` , `tool_manifest_hash` , `config_hash` , `origin_intent_hash`) MUST be computed over a JCS-canonicalized ({{RFC8785}}) JSON representation of the underlying data. Implementations that compute these hashes inconsistently will produce incompatible tokens; canonicalization is normative.

Taint Propagation

Model

Taint represents information provenance. When an agent session ingests content from a source whose trustworthiness is less than the trust level of the session's authorized operations, that ingestion **MUST** be recorded as a taint marker on the session. Taint is monotonic: once applied, it persists for the remainder of the session and is inherited by any sub-agent spawned by that session.

The policy significance of taint is that it reduces the effective capability envelope at policy-decision time. A capability that was authorized in the initial token **MAY** be unavailable in a later turn if taint has accumulated that, by local policy, incompatibly combines with that capability. This is the mechanism by which the "lethal trifecta" (private data + untrusted content + external communication) is prevented at protocol level.

Taint Marker Format

A taint marker is a JSON object with the following fields:

- `id` : Opaque unique identifier for this marker.
- `source` : Source classification URI (e.g., `urn:ctp:taint:source:web-fetch`, `urn:ctp:taint:source:user-upload`, `urn:ctp:taint:source:external-mcp-server`).
- `sensitivity` : Sensitivity tier of the tainting content (implementation-defined).
- `applied_at` : ISO 8601 timestamp.
- `applied_at_turn` : Turn number at which the taint was applied.
- `scope` : Optional. `"session"` (default) or `"turn"`. `"turn"` taints apply only to the current turn and are not propagated; this is reserved for cases where the tainting content is strictly ephemeral. Session-scoped taints (the default) propagate for the remainder of the session.

Taint Application Rules

An authorization server issuing a token for turn N+1 of a session **MUST** include in `ctp_taint` the union of:

1. All session-scoped taints present in the turn N token.

2. All new taint markers corresponding to tool results returned during turn N whose source classification indicates untrusted origin.

A resource server receiving an agent token **MUST** reject the request if any taint marker in `ctp_taint` is incompatible with the requested operation under local policy.

Taint-Aware Policy Decisions

Policy decisions in the CodeTether Profile are attribute-based and **MUST** consider `ctp_taint` alongside the capability envelope. A canonical policy predicate has the form:

```
allow(request) :-  
    request.tool in token.ctp_capability.tools,  
    arguments_valid(request.tool, request.arguments,  
                    token.ctp_capability.tool_constraints),  
    not taint_blocks(request.tool, token.ctp_taint),  
    not budget_exceeded(token.ctp_capability.budget,  
                        session_state(token.ctp_session.id)).
```

The `taint_blocks` predicate is deployment-specific but **MUST** be monotonic: adding a taint marker **MUST NOT** cause a previously blocked operation to become allowed.

Capability Attenuation and Sub-Agent Spawning

The Spawn Exchange

When an agent with token `T_parent` spawns a sub-agent, it **MUST** perform a token exchange per {{RFC8693}} with the following profile-specific constraints:

- `subject_token` **is** `T_parent`.
- `actor_token` **is** the spawning agent's archetype credential.
- `requested_token_type` **is** the agent JWT type registered in Section 11.
- A new parameter `ctp_spawn_request` **is** included, containing:
 - `archetype` : The requested sub-agent archetype identifier.
 - `purpose` : Canonical description of the sub-agent task.
 - `requested_capability` : The capability envelope requested for the sub-agent.

The authorization server **MUST**:

1. Verify `T_parent` is valid, not revoked, and not expired.
2. Verify that `T_parent.ctp_capability.spawn.max_depth` is greater than the current depth of `T_parent`.
3. Verify that the requested archetype is in `T_parent.ctp_capability.spawn.allowed_archetypes`.
4. Verify that `requested_capability` is a subset of `T_parent.ctp_capability` under the attenuation rules of Section 6.2.
5. Verify current fan-out for `T_parent.ctp_session.id` does not exceed `T_parent.ctp_capability.spawn.max_fanout`.
6. Propagate `ctp_taint` from parent to child unchanged.
7. Inherit `ctp_session.origin_intent_hash` unchanged.
8. Issue a new session identifier for the sub-agent with a fresh `parent_session_id` link to the parent session.

Attenuation Rules

Let `P` be the parent capability envelope and `C` the child requested envelope. The authorization server MUST verify:

- `C.tools` is a subset of `P.tools`.
- For each tool `t` in `C.tools`, `C.tool_constraints[t]` is at least as restrictive as `P.tool_constraints[t]`. A constraint `C'` is "at least as restrictive" as `C` if every argument tuple satisfying `C'` also satisfies `C`.
- `C.budget.max_tool_calls` $\leq$ `P.budget.max_tool_calls` - `consumed(P.ctp_session.id)`.
- Similar inequalities hold for all budget fields.
- `C.spawn.max_depth` $\leq$ `P.spawn.max_depth` - 1.
- `C.spawn.max_fanout` $\leq$ `P.spawn.max_fanout`.
- `C.spawn.allowed_archetypes` is a subset of `P.spawn.allowed_archetypes`.

These rules are normative. An authorization server MUST reject a spawn request that violates any of them.

Spawn Depth Enforcement

A token whose `ctp_capability.spawn.max_depth` is 0 MUST NOT be usable as a `subject_token` in a spawn exchange. This provides a hard bound on recursive agent spawning.

Deployments SHOULD set `max_depth` to the minimum value sufficient for the task. Values greater than 4 are discouraged and MUST be accompanied by elevated logging and anomaly detection.

Cross-Trust-Domain Invocation

Problem Statement

When an agent in trust domain A invokes an agent in trust domain B, neither domain's authorization server trusts the other's token issuer. Existing {{RFC8693}} token exchange assumes a single trust domain. This profile defines a bilateral invocation mechanism for A2A calls across domain boundaries.

Cross-Domain Invocation Token (CDIT)

A Cross-Domain Invocation Token is a JWT issued by trust domain A's authorization server, intended for presentation to trust domain B's authorization server. It contains:

- All standard CodeTether Profile claims describing the invoking agent in domain A.
- A new claim `ctp_cross_domain`:
 - `target_domain` : The URI of the target trust domain.
 - `invocation_intent` : Canonical description of the requested action.
 - `attenuation_offer` : The capability envelope the invoker proposes for the cross-domain call.
 - `attestation` : Attestation quote for the invoker's runtime.

The CDIT is signed by domain A's authorization server with a key listed in a federation metadata document that domain B has preconfigured trust in.

Federation Metadata

Each participating trust domain publishes a federation metadata document containing:

- `domain` : The domain's URI.
- `authorization_server` : The AS endpoint.
- `jwks_uri` : JWKS for CDIT signature verification.
- `trusted_domains` : List of trust domains whose CDITs this domain will accept.
- `attestation_verifiers` : List of accepted attestation authorities.
- `archetype_policy` : Per-archetype cross-domain invocation policies.

Federation metadata documents are signed and distributed via a mechanism out of scope for this document. Implementations MAY use OpenID Federation as the distribution substrate.

Inbound CDIT Processing

A trust domain B authorization server receiving a CDIT MUST:

1. Verify the CDIT signature against the issuing domain's JWKS.
2. Verify the issuing domain is in B's `trusted_domains`.
3. Verify the CDIT's attestation quote against B's `attestation_verifiers`.
4. Apply B's local policy to the `invocation_intent`, treating the `attenuation_offer` as the maximum authorized envelope but not the minimum.
5. If authorized, issue a domain-B-local token for the cross-domain call. This token's `act` chain MUST include the full domain-A `act` chain plus a marker indicating the trust-domain boundary.

The domain-B-local token's capability envelope MUST NOT exceed what both B's local policy and the attenuation offer authorize, whichever is more restrictive.

Revocation Across Domains

When a session is revoked in domain A, domain A's authorization server SHOULD notify domain B of any active cross-domain invocations originating from that session. A revocation notification format is defined in Appendix B.

Tool-Call Attestation

Purpose

The profile defined in Sections 4-7 binds tokens to agents, sessions, and delegation chains, but does not yet bind individual tool invocations to specific model outputs. Without this binding, a compromised agent runtime could mint valid tokens for tool calls the model never generated.

Tool-call attestation closes this gap by requiring a signed assertion from the inference runtime that a specific tool call was in fact emitted by the model, at a specific position in the output stream, in response to a specific input context.

Attestation Quote Format

A tool-call attestation quote is a JWT signed by the inference runtime with a key listed in the archetype's attestation authority metadata. It contains:

- `iss` : Inference runtime identifier.
- `aud` : Authorization server that will consume this quote.
- `exp` : Short expiry (RECOMMENDED ≤ 60 seconds).
- `ctp_model` : Model identifier.
- `ctp_system_prompt_hash` : As in Section 4.1.
- `ctp_input_context_hash` : SHA-384 of canonicalized input context at the moment of tool call emission.
- `ctp_output_prefix_hash` : SHA-384 of the model output preceding the tool call.
- `ctp_tool_call` : The complete tool call object as emitted by the model.
- `ctp_runtime_attestation` : Implementation-defined evidence of the runtime environment (e.g., AWS Nitro PCRs, Intel TDX quote, Google Confidential Space attestation). MAY be absent if the inference runtime is a trusted third party attesting at service level rather than hardware level.

Attestation Binding

When a tool-call attestation quote is available, it MUST be included in the token request that the agent runtime makes for the scoped tool token. The authorization server MUST:

1. Verify the quote signature.

2. Verify `ctp_model`, `ctp_system_prompt_hash`, and `ctp_tool_manifest_hash` match the archetype binding of the parent token.
3. Include a reference to the verified quote in the issued tool token as a new claim `ctp_attestation_ref`.

Resource servers MAY require `ctp_attestation_ref` for sensitive operations. Resource servers that require attestation MUST reject tokens without a verified `ctp_attestation_ref`.

Security Considerations

Prompt Injection via Tool Results

This profile does not prevent prompt injection. It limits the blast radius by ensuring that any tool call triggered by injected content is subject to the session's accumulated taint and the capability envelope. Deployments MUST classify tool outputs and apply appropriate taint markers; failure to do so renders the taint mechanism ineffective.

Confused Deputy Across Tools

Capability attenuation (Section 6) and taint propagation (Section 5) together mitigate but do not eliminate confused-deputy attacks. A tool that returns attacker-controlled data influencing a subsequent tool call by the same agent will see that subsequent call as legitimate from an identity perspective. The defense is the taint-aware policy decision at the second tool, not the token itself.

Attestation Trust

The value of tool-call attestation (Section 8) depends entirely on the trustworthiness of the inference runtime issuing the quote. A compromised inference runtime can mint quotes for tool calls the model never produced. Deployments requiring strong assurance SHOULD combine attestation at multiple layers (hardware, hypervisor, inference runtime, orchestrator).

Token Replay Between Sibling Agents

Without proof-of-possession, an agent token could be replayed by a sibling agent in the same multi-agent process. This profile REQUIRES `cnf` (Section 4.1) with either DPoP or mTLS binding. Deployments MUST NOT issue bearer tokens without confirmation

claims.

Denial of Service via Spawn

Unbounded sub-agent spawning is a denial-of-service vector against the authorization server and the downstream resource servers. The `max_depth` and `max_fanout` controls (Section 6) are normative and MUST be enforced at the authorization server.

Clock Skew and Turn Ordering

The `turn` counter (Section 4.1) is an authoritative ordering within a session. Implementations MUST NOT use wall-clock timestamps for ordering decisions where the `turn` counter is sufficient.

Privacy Considerations

Agent tokens contain substantial information about the human principal's activity: the archetype indicates the task type, the origin intent hash is a commitment to the original request, and the `act` chain exposes the delegation graph. Tokens MUST be protected in transit (TLS mandatory) and at rest. Policy decision logs containing token contents are subject to privacy review under the deploying organization's governance framework.

IANA Considerations

JWT Claims Registry

IANA is requested to register the following claims in the JSON Web Token Claims registry:

- `ctp_version`
- `ctp_avatar`
- `ctp_archetype`
- `ctp_session`
- `ctp_capability`
- `ctp_taint`
- `ctp_attestation`

- `ctp_attestation_ref`
- `ctp_cross_domain`
- `ctp_model`
- `ctp_system_prompt_hash`
- `ctp_tool_manifest_hash`
- `ctp_input_context_hash`
- `ctp_output_prefix_hash`
- `ctp_tool_call`
- `ctp_runtime_attestation`

OAuth Parameters Registry

IANA is requested to register the following parameter in the OAuth Parameters registry:

- `ctp_spawn_request`

OAuth Token Type Registry

IANA is requested to register the following token type URI:

- `urn:ietf:params:oauth:token-type:ctp-agent-jwt`

Taint Source URN Namespace

IANA is requested to register the URN namespace `urn:ctp:taint:source:` for taint source identifiers.

— back

CodeTether Operational Notes

This appendix is informative.

The profile defined in this document was derived from operational experience with CodeTether, an autonomous multi-agent development system running A2A-native workers with Protocol Bus coordination, MCP tool integration, and oracle-verified output distillation. Several design decisions are directly traceable to failure modes observed in

that system:

- The `origin_intent_hash` claim was added after observing that long-running sessions can drift from their initial task, allowing a compromised turn to redirect later turns toward attacker-chosen goals. Binding the origin intent at session start and requiring its immutability across turns was necessary to make this drift detectable.
- The monotonic `ctp_taint` model was adopted after attempts with mutable taint resulted in policy-decision inconsistencies when tainted content was ostensibly “cleansed” by intermediate processing. Making taint strictly additive eliminated an entire class of policy bugs.
- The spawn depth and fan-out bounds were introduced after an early forage-cycle implementation spawned sub-agents recursively beyond intended limits due to a prompt-injection vector. Hard bounds at the authorization server made recovery predictable.
- The cross-domain invocation profile reflects the requirements of operating a multi-tenant shared agent service across organizational trust boundaries where per-pair federation configuration is operationally infeasible.
- Tool-call attestation was prototyped using oracle-verified distillation as the attestation substrate. The oracle in CodeTether verifies model outputs against expected invariants and signs the verification; this signature functions as a service-level attestation without requiring hardware attestation primitives.

Revocation Notification Format

This appendix is normative for deployments implementing Section 7.5.

A revocation notification is a signed JWT with:

- `iss` : The revoking domain.
- `aud` : The notified domain.
- `iat` : Notification issue time.
- `ctp_revocation` :
 - `session_id` : Session being revoked.
 - `cascade` : Boolean. If true, all sub-agent sessions descending from this session are also revoked.

- `reason` : Implementation-defined classification.

Receiving domains **MUST** process revocation notifications within 60 seconds of receipt and **MUST** terminate matching sessions.

Example Token

This appendix is informative.

```
{
  "iss": "https://as.domain-a.example/",
  "sub": "spiffe://domain-a.example/avatar/riley-research-01",
  "aud": ["https://tools.domain-a.example/"],
  "exp": 1734567890,
  "iat": 1734567590,
  "jti": "01HX9K...",
  "azp": "spiffe://domain-a.example/archetype/claude-opus-research",
  "act": {
    "sub": "human:riley@agency-a.example",
    "iss": "https://idp.agency-a.example/"
  },
  "ctp_version": "1",
  "ctp_avatar": {
    "id": "spiffe://domain-a.example/avatar/riley-research-01",
    "owner": "human:riley@agency-a.example",
    "class": "research",
    "created_at": "2026-01-15T00:00:00Z"
  },
  "ctp_archetype": {
    "id": "spiffe://domain-a.example/archetype/claude-opus-research",
    "model": "claude-opus-4-7",
    "model_provider": "anthropic",
    "system_prompt_hash": "sha384:...",
    "tool_manifest_hash": "sha384:...",
    "config_hash": "sha384:..."
  },
  "ctp_session": {
    "id": "01HX9J...",
    "origin_intent_hash": "sha384:...",
    "turn": 7,
    "parent_jti": "01HX9J...turn6",
    "started_at": "2026-04-20T14:00:00Z"
  },
  "ctp_capability": {
    "tools": ["doc:read", "web:fetch"],
    "tool_constraints": {
```

```

    "doc:read": {"classification": {"max": "CUI"}},
    "web:fetch": {"domain": {"allowlist": ["*.gov"]}}
  },
  "budget": {
    "max_tool_calls": 50,
    "max_input_tokens": 500000,
    "max_output_tokens": 100000,
    "max_wall_clock_seconds": 3600
  },
  "spawn": {
    "max_depth": 2,
    "max_fanout": 4,
    "allowed_archetypes": [
      "spiffe://domain-a.example/archetype/claude-worker"
    ]
  }
},
"ctp_taint": [
  {
    "id": "01HX9K...",
    "source": "urn:ctp:taint:source:web-fetch",
    "sensitivity": "untrusted-external",
    "applied_at": "2026-04-20T14:12:00Z",
    "applied_at_turn": 3,
    "scope": "session"
  }
],
"cnf": {
  "jkt": "..."
}
}

```

Acknowledgments

This work builds directly on [{{I-D.ietf-wimse-arch}}](#), [{{I-D.goswami-agentic-jwt}}](#), [{{I-D.klrc-aiagent-auth}}](#), and the broader OAuth and WIMSE working group efforts to extend identity protocols for autonomous agent systems. Operational lessons informing this profile arose from deploying autonomous multi-agent systems in regulated environments.